

Royalty Calculation (1)

This document provides technical details for the "Royalty Calculation (1)" Retool web application. The application is designed to manage and calculate royalties based on sales data, contract information, and adjustments, facilitating the generation of royalty reports and billing records.

Application Purpose and Workflow

The primary purpose of this application is to automate and streamline the royalty calculation process. It allows users to:

- **Record and manage sales data:** Input and update sales records, associating them with specific games, licensors, and contracts.
- **Apply royalty adjustments:** Manually adjust calculated royalties to account for various factors.
- **Calculate Minimum Guarantees (MG) and actual royalties:** Determine royalty amounts due based on sales and contract terms, including MG calculations.
- **Generate royalty billing records:** Create records for royalty payments to licensors.
- **Produce royalty reports:** Summarize royalty data for analysis and reporting.
- **Manage licensor transfers:** Facilitate the transfer of contracts between licensors.

The general workflow involves:

1. **Data Input:** Users input or import sales data.
2. **Calculation:** The system calculates royalties based on sales, contract terms, and any applied adjustments. This often involves calculating MG balances.
3. **Review and Adjustment:** Users can review calculated royalties and apply manual adjustments.
4. **Billing and Reporting:** Based on the finalized calculations, billing records are generated, and reports can be produced.
5. **Monthly Closing:** A process exists to close out monthly royalty calculations, likely preparing for the next period.

Major Data Flows

The application interacts with a MySQL database (`NISA_mysql_connection`) to store and retrieve data. Key data flows include:

- **Sales Data Ingestion:** Sales data is inserted into or updated in the `sales` table. This data is then used for royalty calculations.
- **Contract and Licensor Data:** Information from `contract` and `licensor` tables is crucial for applying royalty rates and identifying parties involved.

Royalty Calculation Flow:

- Sales data is aggregated.
- `monthly_royalty` and `monthly_royalty_status` tables are updated with calculated royalty figures.
- `royalty_adjustment` table stores manual adjustments.
- The `calculateMGBalance` JavaScript logic orchestrates fetching sales, contract, and adjustment data to compute MG balances and actual royalties.
- **Billing Record Generation:** Queries like `create_changed_records`, `create_closed_original_records`, and `create_open_original_records` are used to populate billing-related tables based on finalized royalty calculations.
- **Reporting Data Aggregation:** Queries such as `get_accumulated_units_sold`, `get_monthly_royalty_for_royalty_report`, and `get_monthly_adjustments_for_royalty_report` aggregate data from various tables to feed into royalty reports.
- **Licensor Transfer Flow:** Data is read from and written to tables related to contract transfers, updating `contract` and potentially creating history records.

Important SQL Queries

The following SQL queries are central to the application's functionality:

`calc_current_mg_balance:`

- **Responsibility:** Calculates the current Minimum Guarantee (MG) balance for a given contract and date range. This is a core component of royalty calculation.
- **Table Interaction:** Likely reads from `sales`, `contract`, and potentially `monthly_royalty` or related status tables.

`get_mg_balance_for_report:`

- **Responsibility:** Retrieves MG balance data specifically formatted for royalty reports.
- **Table Interaction:** Reads from aggregated or calculated MG balance data.

`get_max_status_id:`

- **Responsibility:** Fetches the maximum `status_id` from `monthly_royalty_status`. This is likely used to determine the latest processed month or status.
- **Table Interaction:** Reads from `monthly_royalty_status`.

`get_most_recent_closed_date:`

- **Responsibility:** Retrieves the most recent date from `monthly_royalty_status` where the status indicates a closed month. Used to establish the end date for subsequent calculations.
- **Table Interaction:** Reads from `monthly_royalty_status`.

`is_status_record_exists:`

- **Responsibility:** Checks if a status record for a specific month and year already exists in `monthly_royalty_status`. Prevents duplicate processing.
- **Table Interaction:** Reads from `monthly_royalty_status`.

`create_changed_records:`

- **Responsibility:** Generates billing records for royalty amounts that have changed from a previous calculation or status.
- **Table Interaction:** Writes to a billing-related table (e.g., `royalty_billing_record`). Reads from `monthly_royalty` and potentially historical status tables.

`create_closed_original_records:`

- **Responsibility:** Creates billing records for original royalty amounts when a month is closed.
- **Table Interaction:** Writes to a billing-related table. Reads from `monthly_royalty`.

`create_open_original_records:`

- **Responsibility:** Creates billing records for original royalty amounts when a month is open.
- **Table Interaction:** Writes to a billing-related table. Reads from `monthly_royalty`.

`get_accumlated_units_sold:`

- **Responsibility:** Calculates the accumulated units sold for a specific game title and contract over a given period. Used in royalty calculations and reporting.
- **Table Interaction:** Reads from `sales`.

`get_contracts_for_royalty_report:`

- **Responsibility:** Fetches contract details relevant for generating royalty reports.
- **Table Interaction:** Reads from `contract`.

`get_monthly_royalty_for_royalty_report:`

- **Responsibility:** Retrieves monthly royalty figures for inclusion in reports.
- **Table Interaction:** Reads from `monthly_royalty`.

`get_monthly_adjustments_for_royalty_report:`

- **Responsibility:** Retrieves monthly royalty adjustments for inclusion in reports.
- **Table Interaction:** Reads from `royalty_adjustment`.

`add_sales:`

- **Responsibility:** Inserts new sales records into the `sales` table.
- **Table Interaction:** Writes to `sales`.

`check_if_record_closed:`

- **Responsibility:** Verifies if a specific sales record has already been processed and closed.

- **Table Interaction:** Reads from `monthly_royalty_status` or a similar status tracking table.

`delete_single_record:`

- **Responsibility:** Deletes a single sales record from the `sales` table.
- **Table Interaction:** Deletes from `sales`.

`get_sales:`

- **Responsibility:** Retrieves sales records, likely with filtering capabilities based on dates, licensors, or games.
- **Table Interaction:** Reads from `sales`.

`update_sales:`

- **Responsibility:** Modifies existing sales records in the `sales` table.
- **Table Interaction:** Updates `sales`.

`get_licensor_contract_for_create:`

- **Responsibility:** Fetches licensor and contract details needed when creating new entries (e.g., new sales records or adjustments).
- **Table Interaction:** Reads from `licensor` and `contract`.

`get_contracts_by_transfer_from_licensor:`

- **Responsibility:** Retrieves contracts associated with a licensor from whom contracts are being transferred.
- **Table Interaction:** Reads from `contract`.

`insert_transfer_contract:`

- **Responsibility:** Records the details of a contract transfer.
- **Table Interaction:** Writes to a contract transfer history table.

`update_active_contract:`

- **Responsibility:** Updates the status or details of an active contract, potentially marking it as transferred.
- **Table Interaction:** Updates `contract`.

`get_active_contracts:`

- **Responsibility:** Fetches a list of currently active contracts.
- **Table Interaction:** Reads from `contract`.

`get_game_titles:`

- **Responsibility:** Retrieves a list of available game titles.
- **Table Interaction:** Reads from a `game_titles` table or similar.

`get_royalty_adjustment_for_calculation_mg:`

- **Responsibility:** Fetches royalty adjustments relevant for MG balance calculations.
- **Table Interaction:** Reads from `royalty_adjustment`.

`get_royalty_due_from_closed_month:`

- **Responsibility:** Retrieves royalty amounts due from a previously closed month.
- **Table Interaction:** Reads from `monthly_royalty` and `monthly_royalty_status`.

`get_royalty_due_from_open_month:`

- **Responsibility:** Retrieves royalty amounts due from an open month.
- **Table Interaction:** Reads from `monthly_royalty`.

`read_licensor:`

- **Responsibility:** Fetches licensor information.

- **Table Interaction:** Reads from `licensor`.

JavaScript Logic

Key JavaScript logic blocks and their roles:

`calculateMGBalance:`

Role: This is a complex, multi-step JavaScript query that orchestrates the calculation of Minimum Guarantee (MG) balances and actual royalties. It involves:

- Fetching the most recent closed date (`get_most_recent_closed_date`).
 - Fetching the maximum status ID (`get_max_status_id`).
 - Fetching sales data (`get_sales`).
 - Fetching contract details (`get_licensor_contract_for_create`).
 - Fetching royalty adjustments (`get_royalty_adjustment_for_calculation_mg`).
 - Calculating accumulated units sold (`get_accumlated_units_sold`).
 - Applying royalty rates and MG thresholds.
 - Determining the royalty amount due.
- **Inference:** This function is likely called when a user initiates a royalty calculation or when data is loaded that requires updated MG balances. It appears to handle both open and closed months by referencing `get_royalty_due_from_closed_month` and `get_royalty_due_from_open_month` indirectly through its internal logic.

`transform_royalty_and_adjustment_table_to_insert_monthly_royalty:`

- **Role:** Formats data retrieved from the `royalty_and_adjustment` table (likely a combined view or query result) into a structure suitable for insertion into the `monthly_royalty` table.
- **Inference:** This transformer is used before inserting calculated royalties.

`mgStatusContracts:`

- **Role:** Calculates recouped and recouping contract statuses based on MG balance data.
- **Inference:** This logic is likely used to display contract status indicators in tables or reports.

`create_report:`

- **Role:** Defines the structure, columns, and potentially styling for generated royalty reports.
- **Inference:** This function is called when a user requests to generate a report, preparing the data and layout.

`delete_record_transform:`

- **Role:** Handles the deletion of sales records. It includes validation to ensure a record is not already closed before deletion.
- **Inference:** This is triggered by a user action to delete a sales record, likely via a button in a sales table.

`get_sales_records_reduce_by_licensor:`

- **Role:** Aggregates sales records, grouping them by licensor or sales account.
- **Inference:** Used for summarizing sales data for reporting or analysis purposes.

`transfer_licensors:`

- **Role:** Manages the process of transferring contracts from one licensor to another. This involves updating contract records and potentially logging the transfer history.
- **Inference:** This logic is triggered by a dedicated UI element for licensor transfers.

`get_credit_records_filtered_by_game_titles, get_credit_records_filtered_by_gl_accounts, get_credit_records_filtered_by_licensors:`

- **Role:** These JavaScript queries filter credit records based on selected game titles, GL accounts, or licensors.
- **Inference:** These are likely used in conjunction with filter widgets (Select widgets, Checkboxes) to refine the display of credit-related data.

`removed_recouped_credit_records:`

- **Role:** Filters out credit records that have already been recouped.
- **Inference:** Used to present only outstanding or relevant credit records.

Key Widgets and Cross-Component Dependencies

TableWidget instances (e.g., `sales`, `royalty_and_adjustment`, `mg_balance`, `mg_table2`, `royalty_table`, `adjustment`, `table2`):

- **Role:** Display tabular data. They are often populated by SQL queries or JavaScript logic.
- **Dependencies:** Their data sources are dependent on the queries and JavaScript logic that fetch and transform data. User interactions within tables (e.g., selecting a row) often trigger other actions or open modals.

SelectWidget2 instances (e.g., for licensors, contracts, game titles, GL accounts):

- **Role:** Provide dropdown selection for filtering or input.
- **Dependencies:** Their `.value` property is frequently used in SQL queries and JavaScript logic to filter data or pass selected IDs to other components. For example, a selected licensor in a `SelectWidget2` might be used in the `WHERE` clause of a `get_sales` query.

DateWidget instances (e.g., `start_date`, `end_date`, `sales_start_date`, `sales_end_date`):

- **Role:** Allow users to select date ranges.
- **Dependencies:** Their `.value` property is used in SQL queries to filter data by date. For example, `sales_start_date.value` and `sales_end_date.value` would be used in a `WHERE` clause for the `sales` table.

ButtonWidget2 instances (e.g., for "Calculate Royalties", "Generate Report", "Add Sales", "Delete Record"):

- **Role:** Trigger actions.
- **Dependencies:** Their `onClick` event handlers are linked to JavaScript queries or SQL queries. For example, a "Calculate Royalties" button's `onClick` event might trigger the `calculateMGBalance` JavaScript query.

FormWidget2 and ContainerWidget2:

- **Role:** Structure the UI, group related widgets, and manage layout.
- **Dependencies:** Contain other widgets and define the visual organization of the application.

CheckboxWidget2 instances (e.g., `is_all_game_titles`):

- **Role:** Provide boolean selection options.
- **Dependencies:** Their `.value` property is often used in conditional logic within JavaScript or SQL queries to determine whether to apply a filter or include all items. For example, if `is_all_game_titles.value` is true, a query might omit the game title filter.

Modals (e.g., `create_sales_modal`, `update_sales_modal`):

- **Role:** Provide interfaces for creating or editing data.
- **Dependencies:** They contain input widgets (`TextInputWidget2`, `NumberInputWidget`, `SelectWidget2`) and buttons that trigger data insertion or update queries. The data displayed within a modal for editing is often pre-populated by a query based on a selected row in a table.

Appendix

Major Queries and Their Responsibilities

| Query Name | Responsibility